

ChaCha20: A Modern Stream Cipher

Sreejit Maity

July 30, 2026

1 Introduction

ChaCha20 is a symmetric stream cipher designed by Daniel J. Bernstein in 2008 as a refinement of his earlier Salsa20 cipher. It has since become one of the most widely deployed ciphers in modern cryptography, used in TLS 1.3, SSH, WireGuard, and the Signal protocol, most often paired with the Poly1305 message authentication code as the AEAD construction **ChaCha20-Poly1305**. Its popularity stems from three properties: it is fast in pure software (no dependence on hardware AES instructions), it has a simple, easily auditable design built entirely from addition, rotation, and XOR, and it has resisted over a decade of cryptanalysis. The algorithm is formally standardized in **RFC 8439**.

2 Design Goals and Building Blocks

ChaCha20 is built entirely from three primitive operations on 32-bit words:

- **Addition** modulo 2^{32}
- **Rotation** of bits (circular left shift)
- **XOR** (exclusive or)

This combination is often called an *ARX* cipher (Add-Rotate-XOR). Because these operations map directly to single CPU instructions, ChaCha20 runs efficiently on virtually any processor without needing specialized cryptographic hardware, unlike AES, which benefits enormously from dedicated AES-NI instructions.

The cipher requires the following inputs:

- A **256-bit key** (32 bytes)
- A **96-bit nonce** (12 bytes) — a value that must never repeat under the same key
- A **32-bit block counter**, allowing the cipher to seek to any block of the keystream

3 The ChaCha20 State and Quarter Round

ChaCha20 operates on a 4×4 matrix of 32-bit words (16 words total, 512 bits), initialized as follows:

$$\begin{pmatrix} c_0 & c_1 & c_2 & c_3 \\ k_0 & k_1 & k_2 & k_3 \\ k_4 & k_5 & k_6 & k_7 \\ b & n_0 & n_1 & n_2 \end{pmatrix}$$

where $c_0 \dots c_3$ are fixed constants derived from the ASCII string "expand 32-byte k", $k_0 \dots k_7$ are the eight 32-bit words of the key, b is the block counter, and $n_0 \dots n_2$ are the three words of the nonce.

The core operation is the **quarter round**, which mixes four words (a, b, c, d) of the state:

$$\begin{aligned} a &+= b; & d \oplus &= a; & d &= \text{ROTL}(d, 16) \\ c &+= d; & b \oplus &= c; & b &= \text{ROTL}(b, 12) \\ a &+= b; & d \oplus &= a; & d &= \text{ROTL}(d, 8) \\ c &+= d; & b \oplus &= c; & b &= \text{ROTL}(b, 7) \end{aligned}$$

A full ChaCha20 round consists of applying the quarter round to the four *columns* of the state matrix, followed by applying it to the four *diagonals*. This pair (column round + diagonal round) is called a **double round**. ChaCha20 performs **10 double rounds**, for a total of **20 rounds** — hence the name.

After the 20 rounds, the original input state is added word-by-word to the mixed state (mod 2^{32}), producing a 64-byte **keystream block**. Encryption (or decryption) is then simply the XOR of this keystream with the plaintext (or ciphertext), one block at a time, incrementing the block counter for each successive 64-byte chunk.

4 C Implementation

The implementation below follows RFC 8439 directly. It consists of three layers: the quarter round, the block function that runs 20 rounds and produces one keystream block, and a wrapper that XORs the keystream against an arbitrary-length buffer.

Listing 1: Core ChaCha20 block function and quarter round in C

```

1 #include <stdint.h>
2 #include <string.h>
3
4 #define ROTL32(x, n) (((x) << (n)) | ((x) >> (32 - (n))))
5
6 static void quarter_round(uint32_t *a, uint32_t *b,
7                          uint32_t *c, uint32_t *d) {
8     *a += *b; *d ^= *a; *d = ROTL32(*d, 16);
9     *c += *d; *b ^= *c; *b = ROTL32(*b, 12);
10    *a += *b; *d ^= *a; *d = ROTL32(*d, 8);
11    *c += *d; *b ^= *c; *b = ROTL32(*b, 7);
12 }
13
14 /* Produces one 64-byte keystream block. */
15 static void chacha20_block(const uint32_t key[8], uint32_t counter,
16                          const uint32_t nonce[3], uint8_t out[64]) {
17     static const uint32_t constants[4] = {
18         0x61707865, 0x3320646e, 0x79622d32, 0x6b206574

```

```

19     };
20
21     uint32_t state[16];
22     state[0] = constants[0]; state[1] = constants[1];
23     state[2] = constants[2]; state[3] = constants[3];
24     for (int i = 0; i < 8; i++) state[4 + i] = key[i];
25     state[12] = counter;
26     state[13] = nonce[0]; state[14] = nonce[1]; state[15] = nonce[2];
27
28     uint32_t working[16];
29     memcpy(working, state, sizeof(working));
30
31     for (int i = 0; i < 10; i++) {
32         /* column rounds */
33         quarter_round(&working[0], &working[4], &working[8], &working
34             [12]);
35         quarter_round(&working[1], &working[5], &working[9], &working
36             [13]);
37         quarter_round(&working[2], &working[6], &working[10], &working
38             [14]);
39         quarter_round(&working[3], &working[7], &working[11], &working
40             [15]);
41         /* diagonal rounds */
42         quarter_round(&working[0], &working[5], &working[10], &working
43             [15]);
44         quarter_round(&working[1], &working[6], &working[11], &working
45             [12]);
46         quarter_round(&working[2], &working[7], &working[8], &working
47             [13]);
48         quarter_round(&working[3], &working[4], &working[9], &working
49             [14]);
50     }
51
52     for (int i = 0; i < 16; i++) {
53         working[i] += state[i];
54         out[4*i + 0] = (uint8_t)(working[i]);
55         out[4*i + 1] = (uint8_t)(working[i] >> 8);
56         out[4*i + 2] = (uint8_t)(working[i] >> 16);
57         out[4*i + 3] = (uint8_t)(working[i] >> 24);
58     }
59 }
60
61 /* Encrypts/decrypts len bytes by XOR-ing with the keystream. */
62 void chacha20_xor(const uint8_t key_bytes[32], uint32_t counter,
63     const uint8_t nonce_bytes[12],
64     const uint8_t *in, uint8_t *out, size_t len) {
65     uint32_t key[8], nonce[3];
66     for (int i = 0; i < 8; i++)
67         key[i] = key_bytes[4*i] | (key_bytes[4*i+1] << 8) |
68             (key_bytes[4*i+2] << 16) | (key_bytes[4*i+3] << 24);
69     for (int i = 0; i < 3; i++)
70         nonce[i] = nonce_bytes[4*i] | (nonce_bytes[4*i+1] << 8) |
71             (nonce_bytes[4*i+2] << 16) | (nonce_bytes[4*i+3] << 24);
72     ;

```

```

64
65     uint8_t block[64];
66     size_t offset = 0;
67     while (offset < len) {
68         chacha20_block(key, counter++, nonce, block);
69         size_t chunk = (len - offset < 64) ? (len - offset) : 64;
70         for (size_t i = 0; i < chunk; i++)
71             out[offset + i] = in[offset + i] ^ block[i];
72         offset += chunk;
73     }
74 }

```

Because encryption is a simple XOR against a generated keystream, **the same function performs decryption** — calling `chacha20_xor` a second time with the ciphertext as input recovers the original plaintext, since XOR is self-inverse.

4.1 What Is Required to Use This Implementation

To actually integrate ChaCha20 into an application, several practical elements are needed beyond the algorithm itself:

- **A secure key.** The 256-bit key must come from a cryptographically secure random source (e.g. `/dev/urandom` on Linux, `CryptGenRandom` on Windows) or a key-derivation function such as HKDF — never a predictable value.
- **A unique nonce per key.** Reusing a (key, nonce) pair to encrypt two different messages catastrophically breaks the cipher, since XOR-ing the two ciphertexts cancels the keystream and leaks the XOR of the two plaintexts. Nonces are typically a counter or randomly generated per message.
- **Message authentication.** ChaCha20 alone provides only confidentiality, not integrity — it does not detect tampering. In practice it is almost always paired with **Poly1305** to form the authenticated encryption scheme **ChaCha20-Poly1305** (as used in TLS 1.3 and WireGuard).
- **Correct byte ordering.** The key, nonce, and counter are interpreted as little-endian words, so implementations must be careful when packing/unpacking bytes on big-endian systems.

5 Security Considerations

ChaCha20 has been extensively analyzed since its publication and no practical attack breaks the full 20-round version. Reduced-round variants (ChaCha8, ChaCha12) trade some security margin for speed and are used in contexts like Linux’s `/dev/urandom` generator, but the standardized 20-round version is what is recommended for general-purpose encryption. Its main security caveat is not the algorithm itself but correct usage: nonce reuse and missing authentication are by far the most common real-world failure modes.

6 Conclusion

ChaCha20 exemplifies a modern approach to cipher design: a small set of simple, hardware-friendly operations (addition, rotation, XOR), repeated many times, produce a keystream that is computationally indistinguishable from random. Its combination of simplicity, auditability, constant-time

behavior (no data-dependent branches or table lookups, making it naturally resistant to timing side-channels), and strong real-world performance has made it a cornerstone of contemporary protocols such as TLS 1.3 and WireGuard.

I have used Claude and various websites and wikipedia to get info and refine this smooth report on ChaCha20.